A02a · Two Integers and a Promise — Practice A2 · 9618 §19.1 Name: _____ Due: _____

This topic is examined **twice** — described on Paper 3, typed on Paper 4, so this homework has both. **Questions 1–6 are written, on this sheet. Questions 7–11 are code, in `A02a_Practice.java`** (listed at the bottom). One set of numbers across both halves. Every subroutine is marked on **what it does when the structure is full or empty**, so never leave those cases out.

1. Trace it. A stack is stored in an array of 5 elements. **`Top`** is initialised to **`−1`**. [4]
Complete the table. Write **`−1`** in the value column if the operation returns nothing useful.

operation	value returned	`Top` afterwards
`Push(A)`	—	0
`Push(B)`	—	
`Push(C)`	—	
`Pop()`		
`Push(D)`	—	
`Pop()`		
`Pop()`		

2. State what is meant by **overflow** and by **underflow**, and say what each subroutine should return when it happens. [3]
overflow

underflow

3. Write it. Write pseudocode for **`Push()`**. It takes one item, returns **FALSE** if the stack is full and **TRUE** if the item was stored. The array holds **`MaxSize`** elements. [4]

4. The one that goes wrong. A **linear** queue is stored in an array of 5 elements, indices 0 to 4. At a certain moment: [3]

```
HeadPointer = 3  TailPointer = 4
```

(a) How many items does the queue currently hold?

(b) **`Enqueue(60)`** is called. State what it returns.

(c) Explain why, even though the array has empty elements.

5. The circular queue. [3]

(a) Write the pseudocode statement that moves **`TailPointer`** on in a **circular** queue. [1]

(b) Making the queue circular creates a new problem: the pointers can no longer tell you whether it is full or empty. Explain why, and state one way to solve it. [2]

6. Choose and justify. State whether a **stack** or a **queue** is the right structure, and give a reason. [2]

(a) a printer serving jobs from several computers

(b) the `undo` feature of a text editor

💻 **QUESTIONS 7–11 — ON THE COMPUTER, NOT ON THIS SHEET.** Open `A02a_Practice.java`. It contains a short unmarked warm-up and then these questions. The file **tests itself** — run it and it tells you which cases you are failing, so keep going until they pass.

7	`Push()` — store an item; FALSE if the stack is full	[4]
8	`Pop()` — return the top item; −1 if the stack is empty	[4]
9	`Enqueue()` — store at the tail; FALSE if full *(and do not forget `HeadPointer`)*	[6]
10	`Dequeue()` — return the head item; −1 if the queue is empty	[5]
11	🍦 the **circular** queue — optional, not marked	—

To hand in: rename the file to `A02a_YourName.java` and submit it. Change nothing inside — the class is not `public`, so Java does not mind what the file is called. · **Written 19 marks + code 19 marks = 38.**

🕳 **Want to dig?** Question 1 asked you for the value *returned* by each `Pop`, and you probably assumed the popped item disappears from the array. It does not. `Pop` only moves `Top` down — the value is still sitting in the element, perfectly readable, until something later overwrites it. So: **after your last `Pop`, what is in element 2?** Then think about what that means for a stack that once held a password.

Additional space. If you need more room for any answer, continue here and write the question number in the margin beside it.